ProjectMatch – Project Structure

Monorepo with two apps: `client` (React) and `server` (Node/Express).

```
projectmatch/
├── README.md
├── architecture.md
├── structure.md
├── implementation.md
├── .gitignore
├── .github/
│   └── workflows/
│       └── ci.yml                # lint + test + build
├── docker-compose.yml            # mongo + redis for local dev
│
├── client/                       # React frontend (Vite)
│   ├── package.json
│   ├── vite.config.js
│   ├── tailwind.config.js
│   ├── .env.example
│   ├── index.html
│   └── src/
│       ├── main.jsx
│       ├── App.jsx
│       ├── routes/
│       │   ├── index.jsx         # route table
│       │   ├── ProtectedRoute.jsx
│       │   └── AdminRoute.jsx
│       ├── api/
│       │   ├── axios.js          # instance + refresh interceptor
│       │   ├── auth.api.js
│       │   ├── users.api.js
│       │   ├── skills.api.js
│       │   ├── projects.api.js
│       │   ├── teams.api.js
│       │   ├── requests.api.js
│       │   ├── matching.api.js
│       │   ├── chat.api.js
│       │   ├── notifications.api.js
│       │   ├── tasks.api.js
│       │   ├── ratings.api.js
```

```
|       |   └── admin.api.js
|       ├── store/
|       |   ├── index.js
|       |   ├── authSlice.js
|       |   ├── notificationSlice.js
|       |   └── chatSlice.js
|       ├── socket/
|       |   ├── socket.js           # singleton client
|       |   └── useSocketEvents.js  # subscribe/unsubscribe hook
|       ├── hooks/
|       |   ├── useAuth.js
|       |   ├── useDebounce.js
|       |   └── useInfiniteScroll.js
|       ├── pages/
|       |   ├── Landing.jsx
|       |   ├── auth/
|       |   |   ├── Login.jsx
|       |   |   ├── Register.jsx
|       |   |   └── VerifyEmail.jsx
|       |   ├── Dashboard.jsx
|       |   ├── profile/
|       |   |   ├── MyProfile.jsx
|       |   |   ├── EditProfile.jsx
|       |   |   └── PublicProfile.jsx
|       |   ├── matching/
|       |   |   └── Recommendations.jsx
|       |   ├── projects/
|       |   |   ├── ProjectList.jsx
|       |   |   ├── ProjectDetail.jsx
|       |   |   └── ProjectForm.jsx
|       |   ├── teams/
|       |   |   ├── MyTeams.jsx
|       |   |   └── TeamWorkspace.jsx   # tabs: overview / chat / tasks
/ members
|       |   ├── requests/
|       |   |   └── Requests.jsx        # invites + applications
|       |   ├── notifications/
```

```
|       |   |   └── Notifications.jsx
|       |   ├── ratings/
|       |   |   └── RateTeammates.jsx
|       |   └── admin/
|       |       ├── AdminDashboard.jsx
|       |       ├── ManageUsers.jsx
|       |       ├── ManageProjects.jsx
|       |       ├── Reports.jsx
|       |       └── ManageSkills.jsx
|       ├── components/
|       |   ├── layout/ (Navbar, Sidebar, Footer, PageShell)
|       |   ├── ui/ (Button, Input, Modal, Badge, Avatar, Skeleton,
Toast)
|       |   ├── profile/ (SkillPicker, AvailabilityEditor, ProfileCard)
|       |   ├── matching/ (MatchCard, MatchScoreBar, WhyMatchPanel)
|       |   ├── projects/ (ProjectCard, RoleSlots, FilterBar)
|       |   ├── chat/ (ConversationList, MessageList, MessageInput,
TypingIndicator)
|       |   ├── tasks/ (KanbanBoard, TaskCard, TaskModal)
|       |   ├── notifications/ (NotificationBell, NotificationItem)
|       |   └── ratings/ (StarRating, ReviewCard)
|       ├── utils/ (formatDate.js, constants.js, validators.js)
|       └── styles/ (index.css)
|
└── server/                         # Node + Express backend
    ├── package.json
    ├── Dockerfile
    ├── .env.example
    ├── src/
    |   ├── server.js               # http server + socket init +
graceful shutdown
    |   ├── app.js                  # express app, middleware, routes
    |   ├── config/
    |   |   ├── env.js              # validated env vars
    |   |   ├── db.js               # mongoose connection
    |   |   ├── redis.js
    |   |   └── matchWeights.js     # scoring weights
```

```
    │   ├── middleware/
    │   │   ├── auth.js            # verify JWT, attach req.user
    │   │   ├── authorize.js       # role + ownership guards
    │   │   ├── validate.js        # zod/joi wrapper
    │   │   ├── rateLimiter.js
    │   │   ├── upload.js
    │   │   └── errorHandler.js
    │   ├── modules/
    │   │   ├── auth/       (auth.routes, auth.controller, auth.service,
auth.validation)
    │   │   ├── users/      (user.model, user.routes, user.controller,
user.service)
    │   │   ├── skills/     (skill.model, skill.routes,
skill.controller, skill.service)
    │   │   ├── matching/
    │   │   │   ├── matching.routes.js
    │   │   │   ├── matching.controller.js
    │   │   │   ├── matching.service.js      # orchestrates pipeline
    │   │   │   ├── embedding.service.js     # create/cached embeddings
    │   │   │   ├── scoring.js               # pure scoring functions
    │   │   │   ├── explain.service.js       # LLM explanations
    │   │   │   └── matchFeedback.model.js
    │   │   ├── projects/   (project.model, routes, controller, service)
    │   │   ├── teams/      (team.model, routes, controller, service)
    │   │   ├── requests/   (request.model, routes, controller, service)
    │   │   ├── chat/       (conversation.model, message.model, routes,
controller, service)
    │   │   ├── notifications/ (notification.model, routes, controller,
service)
    │   │   ├── tasks/      (task.model, routes, controller, service)
    │   │   ├── ratings/    (rating.model, routes, controller, service)
    │   │   └── admin/      (report.model, auditLog.model, routes,
controller, service)
    │   ├── sockets/
    │   │   ├── index.js               # init, auth middleware, rooms
    │   │   ├── chat.handlers.js
    │   │   ├── task.handlers.js
```

```
    │   │   └── presence.handlers.js
    │   ├── jobs/
    │   │   ├── queue.js                # BullMQ setup
    │   │   ├── embedding.job.js
    │   │   ├── email.job.js
    │   │   └── ratingReminder.job.js
    │   ├── services/
    │   │   ├── email.service.js
    │   │   ├── llm.client.js           # provider wrapper
    │   │   └── storage.service.js
    │   ├── utils/
    │   │   ├── ApiError.js
    │   │   ├── asyncHandler.js
    │   │   ├── token.js
    │   │   ├── pagination.js
    │   │   └── logger.js
    │   └── seed/
    │       ├── skills.json
    │       ├── seed.js
    │       └── demoUsers.js
    └── tests/
        ├── unit/       (scoring.test.js, token.test.js)
        ├── integration/(auth.test.js, requests.test.js, teams.test.js)
        └── sockets/    (chat.test.js)
```

Conventions

- **Module pattern:** `routes → controller (HTTP only) → service (business logic) → model`. Controllers never touch the DB directly.
- **Naming:** files in `kebab` / `camel` per folder as shown; React components in `PascalCase`.
- **Errors:** throw `ApiError(status, code, message)`; one central error handler formats the response.
- **Config:** read env only in `config/env.js`; everything else imports from there.
- **Pure functions:** scoring logic lives in `scoring.js` with no I/O, so it is trivial to unit test.

- **Branching:** `main` (prod), `develop` (staging), `feat/*`, `fix/*`.